

Software Requirements Specification (SRS)

Project Name: Lord of Mysteries Community Platform (LOM Social)

Version: 1.0

1. Introduction

1.1 Purpose

The purpose of this application is to provide a centralized platform for Lord of the Mysteries (LOTM) fans to interact socially, discuss theories and chapters, access wiki information, track reading progress, share content, participate in community discussions, and avoid spoilers.

The application aims to replace the need for multiple platforms such as Reddit, Discord, Fandom Wiki, Google Docs, and Facebook Groups by integrating all required features into a single ecosystem.

1.2 Scope

Social Features: User profiles, Posts, Comments, Likes, Notifications, Following system.

Community Features: Forums, Groups, Theory discussions, News.

Knowledge Features: LOTM Wiki, Pathway database, Character database, Chapter summaries, Notes.

Reading Features: Reading progress tracker, Spoiler management, Bookmarks.

1.3 Definitions

LOTM – Lord of the Mysteries

Wiki – Knowledge database

Feed – Social timeline

Spoiler – Information revealing future events

Progress Tracker – User's current reading progress.

2. Overall Description

2.1 Product Perspective

Frontend → Backend API → Database → Storage

2.2 User Classes

Guest: View public information, Browse wiki, Register. Cannot post, comment, or chat.

Registered User: Create posts, Comment, Like, Follow users, Join groups, Save notes, Set reading progress.

Moderator: Remove content, Ban users, Approve wiki edits, Manage reports.

Administrator: Manage entire platform, Create announcements, Manage moderators, Manage tags, Manage wiki.

2.3 Assumptions

User has internet access.

User has an account.

User agrees to community guidelines.

3. Functional Requirements

FR1 Authentication System: Register, Login, Logout, Password reset, Email verification.
FR2 User Profile System: Username, Avatar, Bio, Favorite character, Favorite pathway, Reading progress, Join date, Achievements.
FR3 Feed System: Create posts, Edit posts, Delete posts, Save posts, Pin posts, Share posts.
FR4 Comment System: Comment, Edit comments, Delete comments, Reply to comments, Mention users.
FR5 Reaction System: Like posts, Like comments, Future additional reactions.
FR6 Following System: Follow users, Unfollow users, View followers, View following.
FR7 Notification System: Likes, Comments, Mentions, Followers, Messages, Announcements.
FR8 Forum System: Create threads, Reply to threads, Search threads, Pin threads, Lock threads.
FR9 Group System: Create groups, Join groups, Leave groups, Post inside groups.
FR10 Messaging System: Send messages, Receive messages, View conversations.
FR11 Wiki System: Characters, Pathways, Sequences, Artifacts, Organizations, Locations, Epochs.
FR12 Search System: Users, Posts, Wiki pages, Groups, Forums, Tags.
FR13 Tags System: Character tags, Chapter tags, Pathway tags, Theory tags, Spoiler tags.
FR14 Spoiler System: Mark spoilers, Hide spoilers, Filter content according to reading progress.
FR15 Reading Progress Tracker: Set current chapter, Mark chapters as completed, Track reading history.
FR16 Chapter Summary System: Read summaries, Search summaries, Bookmark summaries.
FR17 Notes System: Create notes, Edit notes, Delete notes, Bookmark notes.
FR18 News System: Official announcements, Donghua news, Author news.
FR19 Moderation System: Delete content, Suspend users, Review reports, Manage spoilers, Manage tags.

4. Non-Functional Requirements

Performance: Page loading less than 2 seconds and API response less than 500 ms.
Scalability: Support 1000+ users.
Availability: 99% uptime.
Security: Password hashing, JWT authentication, HTTPS, Input validation, Rate limiting.
Reliability: Daily backups, Logging, Error handling.
Usability: Mobile responsive, Dark mode, Search functionality.
Maintainability: Modular architecture, API documentation, Unit tests.

5. Database Requirements

The system shall store Users, Posts, Comments, Likes, Notifications, Groups, Messages, Wiki pages, Tags, Reading progress, Reports, and Notes.

6. External Interfaces

Frontend: React and TypeScript (website)
OR React native + flutter (mobile app)
Backend: Python and FastAPI.
Database: PostgreSQL.
Cache: Redis.
Storage: AWS S3 / Cloudinary.

7. Very Far Future Enhancements

Real-time chat, Voice chat, Theory voting, Timeline explorer, Interactive pathway tree, Relationship graph, Achievements, Recommendation engine, AI-powered search, and Community-editable wiki.

Initial Use Cases

UC1 Register
UC2 Login
UC3 Create Post
UC4 Comment
UC5 Follow User
UC6 Read Wiki
UC7 Search
UC8 Set Reading Progress
UC9 View Chapter Summary
UC10 Join Group
UC11 Send Message
UC12 Report Content

Recommended Architecture for the SRS

Presentation Layer → API Layer → Business Layer → Data Access Layer → Database